

Chain-of-Thought Tokens

Giving a fixed-depth transformer room to think one step at a time

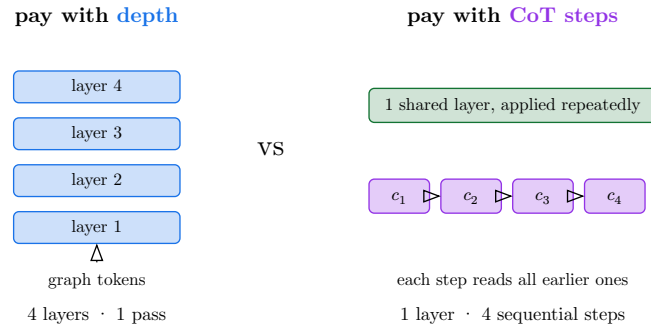
scratchpad reasoning · src/transformer.py · 2026-06-23

In one sentence. A transformer’s forward pass is a **parallel** circuit of fixed depth; chain-of-thought (CoT) tokens let it spend extra **sequential** steps, writing intermediate results into new token positions and reading them back — turning a shallow network into one that can unroll an iterative algorithm like BFS over many rounds.

1 Two budgets: depth and sequential steps

A single forward pass of an L -layer transformer is a bounded-depth computation. Every token is refined L times in parallel; information can cross at most L “hops” of attention before the answer must be read out. For a problem whose answer depends on a long chain of dependencies — **is node s connected to node t ?** — needs reachability across the whole graph — a shallow model simply runs out of rounds.

There are two ways to buy more computation:



Depth is **parallel** time; CoT is **serial** time. Crucially they are interchangeable for iterative problems: a computation that needs D rounds of propagation can be paid for with depth D **or** with D chain-of-thought steps on a shallow model. This is the lever the thesis is missing — Sanford et al.’s depth $\leftrightarrow$ MPC-rounds story covers the first axis; CoT is the second. See [[thesis-sanford-graph-reasoning]].

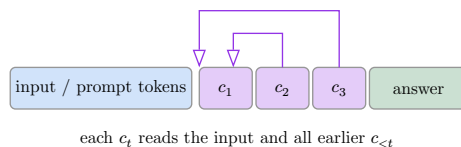
2 What a chain of thought actually is

The idea comes from large language models: prompting a model to “**show its work**” — emit intermediate reasoning before the final answer — sharply improves performance on multi-step problems (Wei et al. 2022; Nye et al. 2021, **scratchpads**). Nothing about the architecture changes. What changes is that the model is allowed to use **its own output positions as a scratchpad**.

The mechanism has two distinct ingredients:

Extra serial computation each generated token is produced by another full forward pass. Ten CoT tokens means ten more passes’ worth of compute spent **before** committing to the answer.

External memory the generated tokens persist in the context. Position t can attend to everything written at positions $< t$. The fixed-width residual stream is a cramped working memory; the growing sequence of tokens is an unbounded tape the model writes to and reads back.



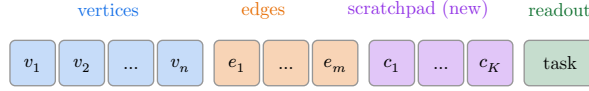
Why this adds real power. Each forward pass is a constant-depth, highly parallel circuit — formally weak (it sits in the complexity class TC^0). One pass cannot, by itself, perform an inherently sequential computation of unbounded length. But a **chain** of T passes composes T such circuits in series, and that composition can simulate a T -step algorithm. Theory makes this precise: transformers with a polynomial number of CoT steps recognise far larger language classes than transformers without (Merrill & Sabharwal 2024; Feng et al. 2023). CoT trades width-bounded parallel weakness for depth-in-time.

3 CoT in a graph transformer

Our `node_edge` model (Sanford-style) already lays each graph out as a sequence and reads the answer from a dedicated task token:

$$\underbrace{[v_1, \dots, v_n]}_{\text{vertex tokens}} + \underbrace{[e_1, \dots, e_m]}_{\text{edge tokens}} + \underbrace{[\text{task}]}_{\text{readout}}$$

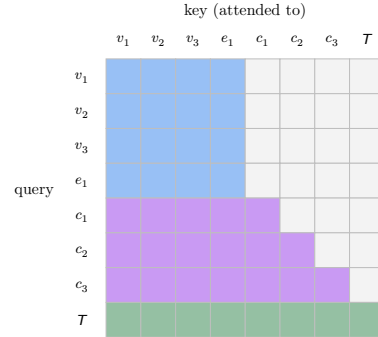
Adding chain-of-thought means inserting K learnable **scratchpad** tokens between the graph and the task token. They carry no input information of their own — they are blank slots the model learns to fill with intermediate state.



3.1 The attention mask is where the algorithm lives

Plain encoder attention is bidirectional — every token sees every token. For the scratchpad to behave like a sequence of computation steps, we constrain who attends to whom. The natural choice mirrors BFS rounds:

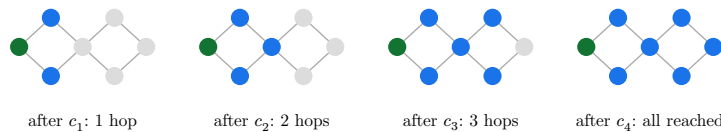
- **Graph tokens** attend among themselves (the fixed problem statement). They **do not** read the scratchpad, so the encoding of the graph stays stable.
- **Scratchpad token c_i** reads all graph tokens **and** every earlier scratchpad token $c_{\leq i}$ — but not later ones. This causal constraint forces c_i to be computed from c_{i-1} , making the slots a genuine sequence of rounds.
- **Task token** reads everything, and the answer is taken from it after the last block.



The lower-triangular **purple** block is the whole trick: it is the causal mask **restricted to the scratchpad**, while the graph stays fully visible to it. Stacking blocks lets each round refine the state further.

3.2 Reading the scratchpad as BFS

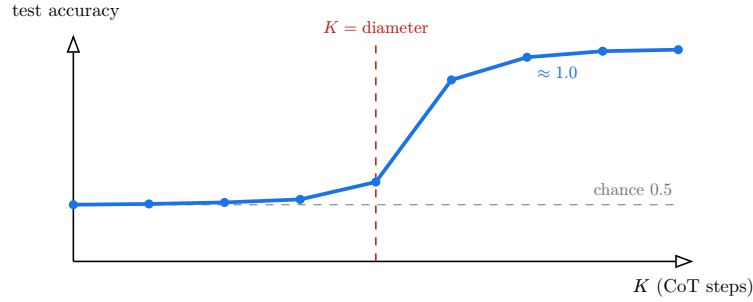
With this mask, the cleanest hypothesis for what the model **can** learn is a frontier expansion. Suppose the task is “is the graph connected from a source s .” Round c_k can hold the set of nodes reachable within k hops; each round attends to the graph and to the previous frontier and grows it by one ring:



The **green** node is the source; **blue** nodes are reached so far; grey nodes are not yet. The reachable set grows by one ring per scratchpad step, so K steps cover everything within distance K . To decide connectivity the model needs $K \geq \text{diameter}$ of the graph — **exactly the prediction we want to test**.

4 The experiment this enables

The phase transition. If CoT genuinely performs frontier expansion, accuracy should be at chance while $K < \text{diameter}$ and jump once $K \geq \text{diameter}$. A model that instead exploits an embedding shortcut would not show this dependence on K at all. That contrast is the experiment.



Controls that make it clean.

- Strip the structural shortcuts: no Laplacian PE, no SPD bias. Otherwise the answer is handed to the model in its features and K becomes irrelevant (the failure mode documented for the connectedness task — see [[connectedness-dataset-leak]]).
- Sweep K on graphs of **known, controlled diameter**; the transition point should track the diameter.
- Compare against a depth sweep at matched parameter count: does one CoT step buy what one extra layer buys? This directly probes the depth $\leftrightarrow$ serial-step equivalence.

5 Supervised vs latent scratchpads

There are two ways to train the scratchpad, with very different difficulty:

	Supervised CoT	Latent CoT
idea	generate ground-truth traces (e.g. the BFS frontier at each round) and train each c_k to predict round k	insert blank scratchpad tokens and train only on the final label; the model self-organises the slots
signal	strong, per-step supervision	weak — gradient only from the answer
cost	must synthesise correct traces; ties the model to one algorithm	trivial to set up; nothing to label
risk	model may copy the trace format without internalising it	optimisation is hard; slots may collapse to no-ops (filler tokens that don't compute)

The latent form is the more interesting scientific test (does sequential capacity **alone** unlock the task?), and it connects to recent results that even content-free **filler** or **pause** tokens can add usable computation when the task structure rewards it (Goyal et al. 2023; Pfau et al. 2024). The supervised form is the stronger baseline if latent training stalls.

6 Implementation in GraphTransformer

The change is small and local to `src/transformer.py`. Three edits:

1. **Append K scratchpad tokens** per graph, between the edge tokens and the task token. A single learnable parameter bank, plus a fourth `type_emb` entry:

```

self.cot = nn.Parameter(torch.randn(config.cot_len, d) * 0.02) # [K, d]
self.type_emb = nn.Embedding(4, d) # vertex / edge / task / cot
# in the per-graph loop, before the task token:
ctok = self.cot + self.type_emb.weight[self.TYPE_COT] # [K, d]
seq = torch.cat([vtok, etok, ctok, ttok], dim=0)

```

2. **Build an additive attention mask** $[L, L]$ encoding the rules above (graph block full, scratchpad lower-triangular over the graph, task row all-visible). `nn.MultiheadAttention` accepts an additive `attn_mask` with $-\infty$ on forbidden entries:

```

mask = torch.zeros(L, L)
mask[is_graph][:, is_cot | is_task] = -inf # graph ignores scratchpad/task
mask[is_cot] = build_causal_over_scratchpad() # see the grid figure
# task row stays all-zero (sees everything)

```

3. **Thread the mask through the blocks.** `_EncoderBlock.forward` currently takes only `key_padding_mask`; add an `attn_mask` argument and pass it to `self.attn(...)`. The readout is unchanged — still `h[task_pos]`.

Two config knobs. `cot_len` (K , the number of scratchpad steps — the swept variable) and a switch for whether blocks **share** the mask (they do; the same causal structure applies at every layer). Keep `lpe_dim: 0` and no SPD so the only way to solve the task is through the scratchpad. With ragged batches, build the mask at the padded length L and combine it with the existing `key_padding_mask`.

7 Pitfalls

- Collapsed slots** with only end-of-sequence supervision, the optimiser may leave the scratchpad as dead weight (the same saddle that stalls anonymous-identity training). Mitigations: curriculum on K , auxiliary per-step losses, or starting from supervised traces.
- Leak re-entry** any positional encoding that already encodes reachability (LPE’s near-zero modes) makes K irrelevant and fakes success. The CoT experiment is only meaningful with the structural shortcuts removed.
- Diameter confound** if graph size and diameter co-vary, a K -dependence might reflect size, not reasoning depth. Control diameter explicitly.
- Attention dilution** long scratchpads add tokens every block; with small width the task token’s attention can wash out. Watch that accuracy gains track K rather than raw parameter count.

8 Summary

Concept	What to remember
CoT token	a blank scratchpad position the model fills with intermediate state; one extra forward pass of serial compute each
depth vs CoT	parallel rounds vs sequential rounds — interchangeable for iterative tasks; CoT is the second axis the depth $\leftrightarrow$ MPC story omits
attention mask	graph full-visible, scratchpad causal over the graph, task sees all — the mask is the algorithm
BFS reading	round c_k holds the k -hop reachable set; connectivity needs $K \geq \text{diameter}$
the test	sweep K ; a phase transition at $K = \text{diameter}$ is evidence of genuine sequential reasoning, not an embedding shortcut
where	<code>src/transformer.py</code> : add scratchpad params, a fourth token type, an <code>attn_mask</code> , and thread it through <code>_EncoderBlock</code>

Chain-of-thought tokens turn the question “**can the transformer reason through connectivity?**” into a measurable one: give it a scratchpad, deny it the structural shortcut, and watch whether the number of thinking steps it needs matches the number of hops the problem actually requires.

References (pointers). Wei et al. 2022, *Chain-of-thought prompting*. Nye et al. 2021, *Show your work: scratchpads*. Feng et al. 2023, *Towards revealing the mystery behind CoT*. Merrill & Sabharwal 2024, *The expressive power of transformers with chain of thought*. Goyal et al. 2023, *Think before you speak: pause tokens*. Pfau et al. 2024, *Let’s think dot by dot: filler tokens*. Sanford et al. 2024a (thesis basis).